

✓ Calculating cost with gradient descent and learning rate

- Change the iteration and learning rate vaules and see the impact on cost.
- Low iteration values with high learning rate (i.e. big steps) may lead to miss the global minimum
- Goal is to reach minimum cost with minimum iteration

 cost_function.png

code credit:codebasics <https://codebasics.io/coming-soon>

import numpy as np

def gradient_descent(x,y):

 m_curr = b_curr = 0

 iterations = 100 #change value

 n = len(x)

 learning_rate = 0.08 #change value

 for i in range(iterations):

 y_predicted = m_curr * x + b_curr

 cost = (1/n) * sum([val**2 for val in (y-y_predicted)])

 md = -(2/n)*sum(x*(y-y_predicted))

 bd = -(2/n)*sum(y-y_predicted)

 m_curr = m_curr - learning_rate * md

 b_curr = b_curr - learning_rate * bd

 print ("m {}, b {}, cost {} iteration {}".format(m_curr,b_curr,cost, i))

x = np.array([1,2,3,4,5])

y = np.array([5,7,9,11,13])

gradient_descent(x,y)

```
m 4.96, b 1.44, cost 89.0 iteration 0
m 0.49919999999999983, b 0.26879999999999993, cost 71.105600000000002 iteration 1
m 4.4515840000000002, b 1.4261760000000001, cost 56.8297702400001 iteration 2
m 0.8922316799999997, b 0.5012275199999995, cost 45.43965675929613 iteration 3
m 4.041314713600002, b 1.432759910400001, cost 36.35088701894832 iteration 4
m 1.2008760606719973, b 0.7036872622079998, cost 29.097483330142282 iteration 5
m 3.7095643080294423, b 1.4546767911321612, cost 23.307872849944438 iteration 6
m 1.4424862661541864, b 0.881337636696883, cost 18.685758762535738 iteration 7
m 3.4406683721083144, b 1.4879302070713722, cost 14.994867596913156 iteration 8
m 1.6308855378034224, b 1.0383405553279617, cost 12.046787238456794 iteration 9
m 3.2221235247119777, b 1.5293810083298451, cost 9.691269350698109 iteration 10
m 1.7770832372205707, b 1.1780607551353204, cost 7.8084968312098315 iteration 11
m 3.0439475772474127, b 1.5765710804477953, cost 6.302918117062937 iteration 12
m 1.8898457226770244, b 1.3032248704973899, cost 5.098330841763168 iteration 13
m 2.898169312926714, b 1.6275829443328358, cost 4.133961682056365 iteration 14
m 1.9761515088959358, b 1.4160484030347593, cost 3.361340532576948 iteration 15
m 2.7784216197824048, b 1.6809279342791488, cost 2.741808050753047 iteration 16
m 2.0415541605113807, b 1.5183370872989306, cost 2.244528230107478 iteration 17
m 2.6796170361078637, b 1.735457156285639, cost 1.8449036666988363 iteration 18
m 2.090471617540917, b 1.611567833948162, cost 1.5233119201782324 iteration 19
m 2.5976890103737853, b 1.790290604096816, cost 1.2640979056612756 iteration 20
m 2.1264168621494517, b 1.6969533824619085, cost 1.0547704368105268 iteration 21
m 2.529385561184701, b 1.8447607474362664, cost 0.8853615531285766 iteration 22
m 2.1521818147302194, b 1.7754939584778073, cost 0.7479156468369821 iteration 23
m 2.472104720735685, b 1.8983676540508527, cost 0.6360820885229722 iteration 24
m 2.1699839382964696, b 1.8480185634495874, cost 0.5447903801652151 iteration 25
m 2.423763296438881, b 1.950743302915348, cost 0.4699911136477278 iteration 26
m 2.1815831093070837, b 1.9152179921582295, cost 0.4084494012702221 iteration 27
m 2.3826922006906663, b 2.0016232209455125, cost 0.35758014655339476 iteration 28
m 2.1883747814212473, b 1.9776712492627107, cost 0.31531667795040486 iteration 29
m 2.3475529664737507, b 2.0508239542984783, cost 0.280005985849834 iteration 30
m 2.19146424741668, b 2.0358666977033213, cost 0.2503251729489924 iteration 31
m 2.317271157065729, b 2.0982251873107836, cost 0.2252148202231392 iteration 32
m 2.19172583072087, b 2.0902190019495084, cost 0.2038258415569305 iteration 33
m 2.2909832477163747, b 2.1437555628915694, cost 0.1854770944836773 iteration 34
m 2.1898500615476015, b 2.1410827139250586, cost 0.16962156815305135 iteration 35
m 2.2679942505397945, b 2.1873814501542004, cost 0.15581941113049289 iteration 36
m 2.1863812735157397, b 2.188763177870427, cost 0.14371641365577967 iteration 37
m 2.247743906750233, b 2.2290980581236037, cost 0.13302683968149862 iteration 38
m 2.181747562970493, b 2.2335252935837153, cost 0.1235197278285518 iteration 39
m 2.2297797112222417, b 2.268922416384484, cost 0.11500795886067308 iteration 40
m 2.176284659606544, b 2.2756005683762908, cost 0.1073395295828815 iteration 41
m 2.213735385878407, b 2.306887840824943, cost 0.10039058653754176 iteration 42
m 2.170254943136438, b 2.315192801071317, cost 0.09405986334903649 iteration 43
m 2.1993136987020745, b 2.3430395801944157, cost 0.08826423771269985 iteration 44
m 2.1638625904931037, b 2.3524826719863134, cost 0.08293518155053264 iteration 45
m 2.1862727486718105, b 2.3774314010318136, cost 0.07801592372722821 iteration 46
m 2.1572656385141533, b 2.3876314575042543, cost 0.07345918129710392 iteration 47
m 2.1744150151272015, b 2.4101229178167802, cost 0.06922534441882239 iteration 48
```

```

m 2.150585587951272, b 2.4207840437050385, cost 0.06528102333197575 iteration 49
m 2.1635786121786147, b 2.441177514495622, cost 0.06159788433500965 iteration 50
m 2.1439150477863547, b 2.4520713783305874, cost 0.05815171649232756 iteration 51
m 2.153630302083688, b 2.470660734860243, cost 0.054921682590988646 iteration 52
m 2.137323817683481, b 2.4816124722824338, cost 0.05188971727120564 iteration 53
m 2.1444599118649865, b 2.4986390442291735, cost 0.04904004275389065 iteration 54
m 2.1308637257526066, b 2.509516039457312, cost 0.04635877856867898 iteration 55
m 2.1359758694885094, b 2.525178884782891, cost 0.04383362645496491 iteration 56
m 2.124572474492945, b 2.535881845863144, cost 0.04145361541183607 iteration 57

```

```

def gradient_descent(x,y):
    m_curr = b_curr = 0
    iterations = 500 #change value
    n = len(x)
    learning_rate = 0.08 #change value

    for i in range(iterations):
        y_predicted = m_curr * x + b_curr
        cost = (1/n) * sum([val**2 for val in (y-y_predicted)])
        md = -(2/n)*sum(x*(y-y_predicted))
        bd = -(2/n)*sum(y-y_predicted)
        m_curr = m_curr - learning_rate * md
        b_curr = b_curr - learning_rate * bd
        print ("m {}, b {}, cost {} iteration {}".format(m_curr,b_curr,cost, i))

x = np.array([1,2,3,4,5])
y = np.array([5,7,9,11,13])

gradient_descent(x,y)

```

```

m 4.96, b 1.44, cost 89.0 iteration 0
m 0.49919999999999983, b 0.26879999999999993, cost 71.105600000000002 iteration 1
m 4.4515840000000002, b 1.4261760000000001, cost 56.8297702400001 iteration 2
m 0.8922316799999997, b 0.5012275199999995, cost 45.43965675929613 iteration 3
m 4.041314713600002, b 1.432759910400001, cost 36.35088701894832 iteration 4
m 1.2008760606719973, b 0.7036872622079998, cost 29.097483330142282 iteration 5
m 3.7095643080294423, b 1.4546767911321612, cost 23.307872849944438 iteration 6
m 1.4424862661541864, b 0.881337636696883, cost 18.685758762535738 iteration 7
m 3.4406683721083144, b 1.4879302070713722, cost 14.994867596913156 iteration 8
m 1.6308855378034224, b 1.0383405553279617, cost 12.046787238456794 iteration 9
m 3.2221235247119777, b 1.5293810083298451, cost 9.691269350698109 iteration 10
m 1.7770832372205707, b 1.1780607551353204, cost 7.8084968312098315 iteration 11
m 3.0439475772474127, b 1.5765710804477953, cost 6.302918117062937 iteration 12
m 1.8898457226770244, b 1.3032248704973899, cost 5.098330841763168 iteration 13
m 2.898169312926714, b 1.6275829443328358, cost 4.133961682056365 iteration 14
m 1.9761515088959358, b 1.4160484030347593, cost 3.361340532576948 iteration 15
m 2.7784216197824048, b 1.6809279342791488, cost 2.741808050753047 iteration 16
m 2.0415541605113807, b 1.5183370872989306, cost 2.244528230107478 iteration 17
m 2.6796170361078637, b 1.735457156285639, cost 1.8449036666988363 iteration 18
m 2.090471617540917, b 1.611567833948162, cost 1.5233119201782324 iteration 19
m 2.5976890103737853, b 1.790290604096816, cost 1.2640979056612756 iteration 20
m 2.1264168621494517, b 1.6969533824619085, cost 1.0547704368105268 iteration 21
m 2.529385561184701, b 1.8447607474362664, cost 0.8853615531285766 iteration 22
m 2.1521818147302194, b 1.7754939584778073, cost 0.7479156468369821 iteration 23
m 2.472104720735685, b 1.8983676540508527, cost 0.6360820885229722 iteration 24
m 2.1699839382964696, b 1.8480185634495874, cost 0.5447903801652151 iteration 25
m 2.423763296438881, b 1.950743302915348, cost 0.4699911136477278 iteration 26
m 2.1815831093070837, b 1.9152179921582295, cost 0.4084494012702221 iteration 27
m 2.3826922006906663, b 2.0016232209455125, cost 0.35758014655339476 iteration 28
m 2.1883747814212473, b 1.9776712492627107, cost 0.31531667795040486 iteration 29
m 2.3475529664737507, b 2.0508239542984783, cost 0.280005985849834 iteration 30
m 2.19146424741668, b 2.0358666977033213, cost 0.2503251729489924 iteration 31
m 2.317271157065729, b 2.0982251873107836, cost 0.2252148202231392 iteration 32
m 2.19172583072087, b 2.0902190019495084, cost 0.2038258415569305 iteration 33
m 2.2909832477163747, b 2.1437555628915694, cost 0.1854770944836773 iteration 34
m 2.1898500615476015, b 2.1410827139250586, cost 0.16962156815305135 iteration 35
m 2.2679942505397945, b 2.1873814501542004, cost 0.15581941113049289 iteration 36
m 2.1863812735157397, b 2.188763177870427, cost 0.14371641365577967 iteration 37
m 2.247743906750233, b 2.2290980581236037, cost 0.13302683968149862 iteration 38
m 2.181747562970493, b 2.2335252935837153, cost 0.1235197278285518 iteration 39
m 2.2297797112222417, b 2.268922416384484, cost 0.11500795886067308 iteration 40
m 2.176284659606544, b 2.2756005683762908, cost 0.1073395295828815 iteration 41
m 2.213735385878407, b 2.306887840824943, cost 0.10039058653754176 iteration 42
m 2.170254943136438, b 2.315192801071317, cost 0.09405986334903649 iteration 43
m 2.1993136987020745, b 2.3430395801944157, cost 0.08826423771269985 iteration 44
m 2.1638625904931037, b 2.3524826719863134, cost 0.08293518155053264 iteration 45
m 2.1862727486718105, b 2.3774314010318136, cost 0.07801592372722821 iteration 46
m 2.1572656385141533, b 2.3876314575042543, cost 0.07345918129710392 iteration 47
m 2.1744150151272015, b 2.4101229178167802, cost 0.06922534441882239 iteration 48
m 2.150585587951272, b 2.4207840437050385, cost 0.06528102333197575 iteration 49
m 2.1635786121786147, b 2.441177514495622, cost 0.06159788433500965 iteration 50
m 2.1439150477863547, b 2.4520713783305874, cost 0.05815171649232756 iteration 51
m 2.153630302083688, b 2.470660734860243, cost 0.054921682590988646 iteration 52
m 2.137323817683481, b 2.4816124722824338, cost 0.05188971727120564 iteration 53
m 2.1444599118649865, b 2.4986390442291735, cost 0.04904004275389065 iteration 54
m 2.1308637257526066, b 2.509516039457312, cost 0.04635877856867898 iteration 55
m 2.1359758694885094, b 2.525178884782891, cost 0.04383362645496491 iteration 56
m 2.124572474492945, b 2.535881845863144, cost 0.04145361541183607 iteration 57

```

```
def gradient_descent(x,y):
    ....m_curr = b_curr = 0
    ....iterations = 100 .....#change value
    ....n = len(x)
    ....learning_rate = 0.01 .....#change value

    ....for i in range(iterations):
    ....y_predicted = m_curr * x + b_curr
    ....cost = (1/n) * sum([val**2 for val in (y-y_predicted)])
    ....md = -(2/n)*sum(x*(y-y_predicted))
    ....bd = -(2/n)*sum(y-y_predicted)
    ....m_curr = m_curr - learning_rate * md
    ....b_curr = b_curr - learning_rate * bd
    ....print ("m {}, b {}, cost {} iteration {}".format(m_curr,b_curr,cost, i))

x = np.array([1,2,3,4,5])
y = np.array([5,7,9,11,13])

gradient_descent(x,y)
```

```
m 2.5439382421747623, b 1.036117596341101, cost 0.7067776678194898 iteration 42
m 2.5421047731158484, b 1.0427589498837933, cost 0.7020065647979179 iteration 43
m 2.540276186037334, b 1.0493774844991666, cost 0.6972676712072055 iteration 44
m 2.5384527760391706, b 1.0559733636469433, cost 0.6925607687282819 iteration 45
m 2.5366347634917363, b 1.0625467298116542, cost 0.6878856408869934 iteration 46
m 2.534822311734855, b 1.0690977094059169, cost 0.6832420728867806 iteration 47
m 2.5330155405888317, b 1.0756264165137073, cost 0.67862985150711 iteration 48
m 2.5312145366684664, b 1.0821329557481032, cost 0.6740487650402432 iteration 49
m 2.5294193612565175, b 1.088617424433033, cost 0.6694986032503796 iteration 50
m 2.5276300563141016, b 1.0950799142689813, cost 0.6649791573458552 iteration 51
m 2.5258466490688605, b 1.1015205126047556, cost 0.6604902199589902 iteration 52
m 2.524069155517426, b 1.1079393034085288, cost 0.6560315851304089 iteration 53
m 2.5222975830990806, b 1.1143363680093126, cost 0.6516030482960009 iteration 54
m 2.520531932736724, b 1.1207117856631814, cost 0.6472044062754403 iteration 55
m 2.518772200394854, b 1.1270656339857144, cost 0.6428354572616448 iteration 56
m 2.517018378268843, b 1.133397989282309, cost 0.6384960008108065 iteration 57
m 2.515270455692759, b 1.1397089268005323, cost 0.6341858378327832 iteration 58
m 2.5135284198323204, b 1.1459985209229562, cost 0.6299047705817177 iteration 59
m 2.5117922562138326, b 1.152266845314558, cost 0.6256526026468309 iteration 60
m 2.510061949127916, b 1.1585139730354368, cost 0.621429138943324 iteration 61
m 2.5083374819376485, b 1.1647399766270532, cost 0.6172341857033854 iteration 62
m 2.506618837313743, b 1.1709449281782531, cost 0.6130675504672674 iteration 63
m 2.504905997414024, b 1.1771288993758635, cost 0.6089290420744439 iteration 64
m 2.503198944020387, b 1.1832919615435047, cost 0.6048184706548296 iteration 65
m 2.5014976586432915, b 1.1894341856714115, cost 0.6007356476200627 iteration 66
m 2.4998021226014826, b 1.1955556424393858, cost 0.596680385654852 iteration 67
m 2.4981123170827932, b 1.201656402234509, cost 0.5926524987083781 iteration 68
m 2.496428223190508, b 1.2077365351648512, cost 0.5886518019857576 iteration 69
m 2.494749821978705, b 1.2137961110701236, cost 0.5846781119395672 iteration 70
m 2.4930770944791827, b 1.2198351995299987, cost 0.5807312462614158 iteration 71
m 2.4914100217219626, b 1.2258538698706478, cost 0.5768110238735877 iteration 72
m 2.489748584750892, b 1.2318521911699172, cost 0.5729172649207284 iteration 73
m 2.4880927646355007, b 1.2378302322614654, cost 0.5690497907615985 iteration 74
m 2.4864425424800025, b 1.243788061738106, cost 0.5652084239608729 iteration 75
m 2.4847978994301156, b 1.2497257479545438, cost 0.5613929882810045 iteration 76
m 2.4831588166782175, b 1.255643359029646, cost 0.5576033086741354 iteration 77
m 2.481525275467231, b 1.26154096284836, cost 0.5538392112740665 iteration 78
m 2.4798972570935383, b 1.267418627063359, cost 0.5501005233882829 iteration 79
m 2.4782747429091585, b 1.2732764190964794, cost 0.546387073490029 iteration 80
m 2.476657714323355, b 1.2791144061400004, cost 0.5426986912104383 iteration 81
m 2.475046152803817, b 1.284932655157799, cost 0.5390352073307189 iteration 82
m 2.4734400398775094, b 1.290731232886414, cost 0.5353964537743896 iteration 83
m 2.4718393571312727, b 1.2965102058360352, cost 0.5317822635995677 iteration 84
m 2.4702440862122303, b 1.302269640291438, cost 0.528192470991312 iteration 85
m 2.4686542088280534, b 1.3080096023128753, cost 0.5246269112540108 iteration 86
m 2.4670697067471092, b 1.3137301577369347, cost 0.5210854208038325 iteration 87
m 2.465490561798529, b 1.3194313721773694, cost 0.5175678371612157 iteration 88
m 2.4639167558722104, b 1.3251133110259103, cost 0.5140739989434152 iteration 89
m 2.4623482709187696, b 1.3307760394530594, cost 0.5106037458571004 iteration 90
m 2.4607850889494567, b 1.336419622408872, cost 0.5071569186909993 iteration 91
m 2.459227192036044, b 1.3420441246237271, cost 0.5037333593085942 iteration 92
m 2.4576745623106904, b 1.34764961060909, cost 0.5003329106408669 iteration 93
m 2.456127181965793, b 1.3532361446582668, cost 0.49695541667909215 iteration 94
m 2.4545850332538226, b 1.358803790847154, cost 0.4936007224676837 iteration 95
m 2.4530408984871526, b 1.3643526130349815, cost 0.4902686740970767 iteration 96
m 2.45151636003788, b 1.3698826748650528, cost 0.48695911869667574 iteration 97
m 2.4499898003376432, b 1.375394039765479, cost 0.48367190442783614 iteration 98
m 2.4484684018774328, b 1.3808867709499106, cost 0.4804068804768991 iteration 99
```

```
def gradient_descent(x,y):
    ....m_curr = b_curr = 0
    ....iterations = 100 .....#change value
    ....n = len(x)
    ....learning_rate = 0.05 .....#change value

    ....for i in range(iterations):
    ....y_predicted = m_curr * x + b_curr
```

```

cost = (1/n) * sum([val**2 for val in (y-y_predicted)])
md = -(2/n)*sum(x*(y-y_predicted))
bd = -(2/n)*sum(y-y_predicted)
m_curr = m_curr - learning_rate * md
b_curr = b_curr - learning_rate * bd
print ("m {}, b {}, cost {} iteration {}".format(m_curr,b_curr,cost, i))

x = np.array([1,2,3,4,5])
y = np.array([5,7,9,11,13])

gradient_descent(x,y)

```

```

m 2.302289804997286, b 1.9086378879240764, cost 0.2243157343632518 iteration 42
m 2.2971796531230484, b 1.9270871576324828, cost 0.21679580768794726 iteration 43
m 2.2921558873979504, b 1.94522454593232, cost 0.20952797789457814 iteration 44
m 2.287217047480509, b 1.963055325119703, cost 0.2025037937254882 iteration 45
m 2.282361697716038, b 1.9805846783635799, cost 0.19571508724170383 iteration 46
m 2.2775884267193223, b 1.99781777012124105, cost 0.1891539643249979 iteration 47
m 2.2728958469643445, b 2.014759403075373, cost 0.18281279549836707 iteration 48
m 2.2682825943809535, b 2.031414708678532, cost 0.1766842070542369 iteration 49
m 2.263747327958345, b 2.047788459496393, cost 0.17076107248009 iteration 50
m 2.2592887293552475, b 2.0638854151592505, cost 0.16503650417153282 iteration 51
m 2.2549055025167, b 2.079710254836751, cost 0.1595038454231776 iteration 52
m 2.2505963732973044, b 2.095267578598066, cost 0.1541566626880198 iteration 53
m 2.24636008909085, b 2.110561908749068, cost 0.14898873809630875 iteration 54
m 2.2421954184661947, b 2.1255976911469063, cost 0.14399406222521696 iteration 55
m 2.2381011508093085, b 2.140379296492357, cost 0.13916682711089676 iteration 56
m 2.234076095971362, b 2.154911021600329, cost 0.13450141949479977 iteration 57
m 2.230119083922765, b 2.1691970906488875, cost 0.12999241429640668 iteration 58
m 2.2262289644130573, b 2.1832416564071693, cost 0.12563456830477537 iteration 59
m 2.2224046066365437, b 2.197048801442535, cost 0.12142281408157163 iteration 60
m 2.218644898903585, b 2.2106225393073187, cost 0.11735225406849686 iteration 61
m 2.214948748317446, b 2.223966815705511, cost 0.11341815489225414 iteration 62
m 2.2113150804566017, b 2.237085509639726, cost 0.10961594186043476 iteration 63
m 2.207742839062422, b 2.2499824345387727, cost 0.10594119364192585 iteration 64
m 2.204230985732126, b 2.262661339366169, cost 0.1023896371256476 iteration 65
m 2.200778499616937, b 2.275125909709914, cost 0.09895714245164931 iteration 66
m 2.197384377125332, b 2.2873797688538415, cost 0.09563971820877826 iteration 67
m 2.1940476316313147, b 2.299426478830858, cost 0.09243350679334461 iteration 68
m 2.190767293187611, b 2.311269541458378, cost 0.08933477992338035 iteration 69
m 2.18754208243725, b 2.3229123993562566, cost 0.08633993430327605 iteration 70
m 2.1843720393687502, b 2.334358436947513, cost 0.08344548743375868 iteration 71
m 2.181255264978871, b 2.3456109814421366, cost 0.08064807356233224 iteration 72
m 2.1781911790694717, b 2.3566733038042615, cost 0.07794443976947857 iteration 73
m 2.175178890951774, b 2.367548619702994, cost 0.0753314421860593 iteration 74
m 2.1722175249939246, b 2.3782400904471626, cost 0.07280604233752795 iteration 75
m 2.1693062203664586, b 2.3887508239042687, cost 0.07036530361069966 iteration 76
m 2.1664441307920734, b 2.399083875403904, cost 0.06800638783896373 iteration 77
m 2.1636304242996216, b 2.4092422486258918, cost 0.0657265520019769 iteration 78
m 2.16086428298227, b 2.419228896473416, cost 0.06352314503599484 iteration 79
m 2.1581449027597484, b 2.4290467219313934, cost 0.0613936047511307 iteration 80
m 2.155471493144607, b 2.4386985789103295, cost 0.05933545485196432 iteration 81
m 2.1528432770124404, b 2.4481872730759147, cost 0.057346302058027765 iteration 82
m 2.1502594903759817, b 2.4575155626645913, cost 0.0554238332082406 iteration 83
m 2.1477193821630243, b 2.4666861592853375, cost 0.053565813134143936 iteration 84
m 2.145222213998096, b 2.4757017287078966, cost 0.05177008093454941 iteration 85
m 2.1427672599878216, b 2.484564891637678, cost 0.05003454858900295 iteration 86
m 2.1403538065099146, b 2.4932782244775638, cost 0.04835719796672347 iteration 87
m 2.1379811520057395, b 2.501844260076833, cost 0.04673607859243991 iteration 88
m 2.135648606776376, b 2.510265488467428, cost 0.04516930537831803 iteration 89
m 2.133355492782134, b 2.518544357587772, cost 0.043655056431923134 iteration 90
m 2.131101143445455, b 2.526683273994355, cost 0.04219157093766536 iteration 91
m 2.128884903457148, b 2.534684603561283, cost 0.040777147109271154 iteration 92
m 2.1267061285859, b 2.5425506721680105, cost 0.039410140210890766 iteration 93
m 2.124564185491007, b 2.5502837663754394, cost 0.0380889606445504 iteration 94
m 2.122458451538267, b 2.557886134090593, cost 0.03681207210171745 iteration 95
m 2.1203883146189955, b 2.5653599852200535, cost 0.035577989776834094 iteration 96
m 2.118353172972084, b 2.5727074923123494, cost 0.03438527864073859 iteration 97
m 2.1163524350090865, b 2.579930791189489, cost 0.03323255177196899 iteration 98
m 2.1143855191422443, b 2.587031981567814, cost 0.03211846874400929 iteration 99

```

```

def gradient_descent(x,y):
    m_curr = b_curr = 0
    iterations = 100    #change value
    n = len(x)
    learning_rate = 0.02    #change value

    for i in range(iterations):
        y_predicted = m_curr * x + b_curr
        cost = (1/n) * sum([val**2 for val in (y-y_predicted)])
        md = -(2/n)*sum(x*(y-y_predicted))
        bd = -(2/n)*sum(y-y_predicted)
        m_curr = m_curr - learning_rate * md
        b_curr = b_curr - learning_rate * bd
        print ("m {}, b {}, cost {} iteration {}".format(m_curr,b_curr,cost, i))

x = np.array([1,2,3,4,5])
y = np.array([5,7,9,11,13])

```



```
gradient_descent(x,y)
```

```
m 2.4700114331944047, b 1.3031095923370972, cost 0.531268423520549 iteration 42
m 2.466833251508415, b 1.3145838366602847, cost 0.524107921409543 iteration 43
m 2.463676560445478, b 1.3259804930128636, cost 0.5170439294395728 iteration 44
m 2.460541214687924, b 1.3373000860388917, cost 0.5100751468349128 iteration 45
m 2.4574270699005702, b 1.348543136834785, cost 0.5032002903518572 iteration 46
m 2.4543339827241453, b 1.3597101629733253, cost 0.49641809404241655 iteration 47
m 2.4512618107687225, b 1.3708016785274948, cost 0.4897273090212088 iteration 48
m 2.4482104126071853, b 1.3818181940941483, cost 0.483126703235482 iteration 49
m 2.445179647768726, b 1.3927602168175202, cost 0.4766150612382476 iteration 50
m 2.442169376732384, b 1.4036282504125723, cost 0.4701911839644611 iteration 51
m 2.4391794609206263, b 1.4144227951881834, cost 0.4638538885102291 iteration 52
m 2.4362097626929686, b 1.4251443480701809, cost 0.45760200791498234 iteration 53
m 2.433260145339641, b 1.4357934026242174, cost 0.45143439094659055 iteration 54
m 2.430330473075293, b 1.4463704490784919, cost 0.4453499018893776 iteration 55
m 2.427420611032745, b 1.456875974346317, cost 0.4393474203349813 iteration 56
m 2.424530425256779, b 1.467310462048535, cost 0.43342584097604553 iteration 57
m 2.4216597826979718, b 1.4776743925357803, cost 0.42758407340268434 iteration 58
m 2.4188085512065705, b 1.4879682429105925, cost 0.4218210419016897 iteration 59
m 2.415976599526408, b 1.4981924870493804, cost 0.41613568525845435 iteration 60
m 2.413163797288863, b 1.5083475956242363, cost 0.4105269565615509 iteration 61
m 2.4103700150068548, b 1.5184340361246031, cost 0.40499382300995795 iteration 62
m 2.4075951240688864, b 1.5284522728787964, cost 0.399535265722871 iteration 63
m 2.4048389967331207, b 1.5384027670753782, cost 0.39415027955209186 iteration 64
m 2.402101506121502, b 1.5482859767843886, cost 0.388837872896933 iteration 65
m 2.3993825262139143, b 1.558102356978433, cost 0.3835970675216255 iteration 66
m 2.39668193184238, b 1.5678523595536258, cost 0.3784268983751844 iteration 67
m 2.3939995986852978, b 1.5775364333503952, cost 0.3733264134137022 iteration 68
m 2.3913354032617193, b 1.5871550241741437, cost 0.368294673425038 iteration 69
m 2.3886892229256658, b 1.5967085748157717, cost 0.3633307518558684 iteration 70
m 2.3860609358604803, b 1.6061975250720608, cost 0.3584337346410733 iteration 71
m 2.3834504210732215, b 1.6156223117659207, cost 0.35360272003541443 iteration 72
m 2.3808575583890934, b 1.6249833687664974, cost 0.34883681844748926 iteration 73
m 2.378282228445913, b 1.6342811270091464, cost 0.34413515227591934 iteration 74
m 2.3757243126886136, b 1.643516014515271, cost 0.33949685574774674 iteration 75
m 2.373183693363791, b 1.6526884564120266, cost 0.33492107475901006 iteration 76
m 2.37066025351428, b 1.6617988749518906, cost 0.33040696671746606 iteration 77
m 2.3681538769737696, b 1.6708476895321014, cost 0.325953700387438 iteration 78
m 2.365664448361459, b 1.679835316713965, cost 0.3215604557367435 iteration 79
m 2.3631918530767413, b 1.6887621702420315, cost 0.317226423785699 iteration 80
m 2.360735977293931, b 1.6976286610631413, cost 0.31295080645814943 iteration 81
m 2.3582967079570243, b 1.7064351973453438, cost 0.3087328164345099 iteration 82
m 2.355873932774492, b 1.715182184496687, cost 0.30457167700678567 iteration 83
m 2.353467540214113, b 1.7238700251838805, cost 0.30046662193555157 iteration 84
m 2.351077419497838, b 1.7324991193508317, cost 0.29641689530885124 iteration 85
m 2.348703460596689, b 1.741069864237058, cost 0.29242175140300447 iteration 86
m 2.346345554225699, b 1.749582654395973, cost 0.2884804545452885 iteration 87
m 2.3440035918388746, b 1.7580378817130502, cost 0.2845922789784686 iteration 88
m 2.341677465624204, b 1.7664359354238632, cost 0.28075650872715713 iteration 89
m 2.3393670684986905, b 1.7747772021320043, cost 0.27697243746597183 iteration 90
m 2.337072294103426, b 1.7830620658268812, cost 0.2732393683894715 iteration 91
m 2.334793036798693, b 1.7912909079013948, cost 0.2695566140838465 iteration 92
m 2.3325291916591007, b 1.799464107169496, cost 0.26592349640033686 iteration 93
m 2.3302806544468757, b 1.807582039883624, cost 0.2623393463303539 iteration 94
m 2.328047321716469, b 1.8156450797520283, cost 0.2588035038822929 iteration 95
m 2.3258290905909793, b 1.823653597955971, cost 0.25531531795999707 iteration 96
m 2.3236258589762318, b 1.8316079631668145, cost 0.25187414624286397 iteration 97
m 2.321437525446672, b 1.8395085415629941, cost 0.24847935506757024 iteration 98
m 2.319263989262577, b 1.8473556968468738, cost 0.24513031931138451 iteration 99
```

✓ Conclusion

This exercise demonstrated the importance of selecting appropriate values for both the learning rate and the number of iterations when using gradient descent. Increasing the number of iterations from 100 to 500 allowed the model to continue reducing the cost function until it was almost zero, although the improvements became much smaller after the first 100 iterations.

Changing the learning rate had a significant impact on the speed of convergence. A learning rate of 0.01 resulted in slow but stable learning, with the cost remaining relatively high after 100 iterations. Increasing the learning rate to 0.20 and 0.50 reduced the cost more quickly, demonstrating that larger parameter updates can accelerate learning. However, the default learning rate of 0.08 achieved the lowest final cost after 100 iterations, suggesting it provided the best balance between convergence speed and optimisation accuracy for this dataset.

Overall, this activity reinforced my understanding of gradient descent and showed that there is no single learning rate that is optimal for every problem. Selecting appropriate hyperparameters is an important part of training machine learning models, as they directly affect both the efficiency of the optimisation process and the quality of the final model.

Start coding or [generate](#) with AI.

